

LEANPOLISH: VERIFIED SUPERVISION FOR LEAN PROOF COMPRESSION

Pauline Bourigault
Imperial College London *

ABSTRACT

Verified proof edits offer a natural source of supervision for improving language-model-generated Lean proofs. Yet verification establishes that an edit is correct, not that its training signal is free of search artifacts. We introduce LEANPOLISH, a symbolic Lean 4 pipeline that releases 33,402 accepted local edits and 65,596 same-state failed attempts, and use it to study what models learn from this supervision. First-success search admits a goal-independent rule with perfect ranking accuracy; teacher-selected evaluation sites also reward trivial deletions. Continuing menu evaluation beyond the first success removes the ordering shortcut: a trained ranker selects the best candidate on 70.1% of evaluated held-out states, versus 36.9% for the strongest frozen baseline. For compression, iterating the symbolic pass raises miniF2F savings from 19.7% to 27.5%, exceeding the neural hybrids we test there. Verified neural editing helps on other proof sources, but matched frozen-model controls show that its gains need not come from training. The supervision does improve whole-proof rewriting: fine-tuning raises verified token reduction from 2.8% to 5.5% on 19 PutnamBench proofs. Together, the released edits, complete candidate pools, and controlled evaluations separate learning to imitate a search policy from improving on that search. They provide a reproducible basis for studying proof improvement while keeping correctness, compression, and edit policy distinct.

1 INTRODUCTION

Language-model provers now produce kernel-checked Lean 4 (de Moura & Ullrich, 2021) proofs for competition problems at scale (Ren et al., 2025; Lin et al., 2025b; Wang et al., 2025; Chen et al., 2025; Achim et al., 2025; Axiom Math, 2025). Correctness does not make these proofs concise: search can leave speculative tactic cascades, unused facts, and repeated derivations. Recent systems therefore learn to shorten proofs using verified outputs of search (Gu et al., 2026; Ahuja et al., 2026; Lu et al., 2026). Such supervision has an appealing intuition: a complete proof shows that one derivation works, whereas a verified edit shows how to improve that derivation in a specific context.

The difficulty is that the search also determines which contexts, alternatives, and labels enter the dataset. A kernel certificate answers “*does this edit preserve the theorem?*”; it does not answer “*what must a model learn to predict this edit?*” If search stops at its first success, its logged failures reveal the winner’s position in the search order. If evaluation uses only sites where the search found an edit, it measures conditional imitation rather than autonomous proof improvement. These effects can make a verified dataset look more informative than its evaluation establishes.

We study this distinction with LEANPOLISH, a symbolic pipeline that replaces goal-closing tactics, removes unused and unreachable material, and factors repeated local facts (Figure 1, Algorithm 1). Its central output is a replayable local record: an original span, a verified replacement, the available proof state, and alternatives tried at that state. This granularity makes the teacher’s decisions inspectable. We use it to ask three questions: *what signal do the records contain, how should that signal be evaluated, and when does learning from it help beyond running the symbolic search?*

*Code: <https://github.com/paulinebourigault/leanpolish>. Dataset: <https://huggingface.co/datasets/leanpolish-anon/lean-proof-compression>.

The controls change the interpretation of apparently strong results. A ranker reaches 100% on first-success groups, but so does a rule that ignores the goal and picks the latest candidate in the fixed menu. A fine-tuned 7B editor recovers 92–98% of the teacher’s savings at teacher-selected sites, yet 40% of these sites are deletions identified by a trivial prompt marker. On tactic-replacement sites, fine-tuning reaches 42–79% success, versus 0–18% with four-shot prompting. Recording complete menu outcomes removes the ordering shortcut; on the resulting evaluation pools, a trained ranker reaches 70.1% top-1 accuracy, compared with 51.0% without the goal and 36.9% for the strongest frozen baseline.

Learning this signal and improving a complete proof are different tests. Iterating LEANPOLISH to a fixed point raises miniF2F token reduction from 19.7% to 27.5%, exceeding all tested neural hybrids on that source. Neural proposals add on PutnamBench and AxiomProver, but a matched frozen editor often achieves as much as a trained one, and some trained edits relax the teacher’s specificity policy. The clearest training benefit is whole-proof rewriting: LEANPOLISH proof pairs raise verified reduction from 2.8% to 5.5% on PutnamBench. In short, the kernel decides which edits are correct, but the search decides which edits are recorded; LEANPOLISH makes that second decision explicit, so it can be controlled, removed, and tested.

Contributions.

1. **A neurosymbolic method for verified proof compression:** a symbolic Lean 4 pass whose every edit is kernel-checked and logged with its proof state and failed alternatives; a complete-menu mode that labels every candidate; and a verified hybrid (Alg. 2) in which neural proposals are admitted only after joint re-verification (§3, §5). Its release comprises 33,402 accepted edits, 65,596 linked failures, and complete pools for about 60,000 proof states.
2. **An evaluation protocol that exposes search selection effects:** menu-order, deletion, prompting, and goal-ablation controls distinguish shortcuts from learnable state-dependent signal (§4).
3. **A controlled study of downstream utility:** whole-file verification, symbolic fixed points, frozen editors, and policy-matched comparisons distinguish gains from supervision from gains due to additional search or a different edit policy (§5).

Together, these contributions make the symbolic teacher, its training signal, and its downstream utility independently inspectable.

2 RELATED WORK

Proof optimization. Lean provers trained on binary success (Polu et al., 2023; Lample et al., 2022; Xin et al., 2024; Ren et al., 2025; Lin et al., 2025b; Wang et al., 2025; Chen et al., 2025; Achim et al., 2025; Hubert et al., 2025) produce long proofs, and several systems now shorten them. ImProver (Ahuja et al., 2025), Lean Refactor (Lu et al., 2026) and Proof-Refactor (Fu et al., 2026) prompt frozen LLM agents; Lean Refactor retrieves strategies distilled from 200K verified long–short pairs. ProofOptimizer (Gu et al., 2026) and ImProver 2 (Ahuja et al., 2026) train 7B models by expert iteration, with RL and preference optimization, respectively, to rewrite *whole* proofs. ProofOptimizer reports 87.9%/57.2% reductions on its Goedel-Prover-V2 miniF2F/PutnamBench inputs; these use different proof samples and aggregation from ours (App. A). On the symbolic side, Mathlib’s linters, `simplifyHint`, and AXLE’s `simplifyTheorems` (Xin et al., 2026), which removes unused tactics and `have`s to a fixed point, overlap with our deletion phases. What differs here is the output and the granularity: LEANPOLISH adds goal-closing tactic replacement under a specificity filter and local-fact anti-unification (Plotkin, 1970; Cerna & Kutsia, 2023), and emits each accepted change as a *local*, *goal-conditioned* edit with the same-state failures, so that a model is trained on a symbolic teacher’s edits rather than on its own whole-proof samples. These systems build optimizers; we study the verified supervision such optimizers are trained on. Their expert iteration and winner/loser pairs are search-generated data of the kind whose selection effects we measure, and our controls (iterated symbolic baseline, matched frozen models, policy compliance) separate gains due to training from gains due to verification and search. The approaches are complementary: LEANPOLISH can pre-process their inputs (12.5% $\rightarrow$ 27.6% for a frozen rewriter on miniF2F-99) and supply complete candidate pools.

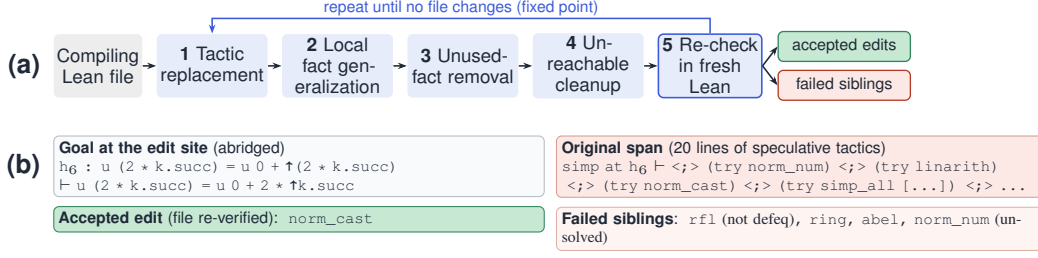


Figure 1: (a) The LEANPOLISH pipeline: candidate tactics are tested at the saved proof state, edited files are re-checked, and fixed-point iteration re-applies the pipeline to its output. Complete-menu mode records all menu outcomes. (b) One released record (miniF2F aime_1984_p1): a learner sees the goal and the original span and must produce the edit; the failed siblings are stored with it.

Proof repair and local edits. Repair conditions on a failed proof and its error (First et al., 2023; Ospanov et al., 2025; Lin et al., 2025b); APRIL (Wang et al., 2026) releases 260K state-conditioned repair tuples from synthetic mutations, and BlueprintRepair (Khrulev, 2026) restricts repair to typed local operations with accepted and rejected trajectories. ProofAug (Liu et al., 2025) tries automation at every sub-proof of an LLM proof, as our tactic menu does, but to find proofs rather than shorten them. Our edits act on *correct* proofs, and our negatives are natural failures of real automation on real goals, not mutations; our hybrid (§5) reuses the verify-splice-recombine loop of APOLLO for compression.

Learning from search and from failures. Training on the verified output of search is expert iteration (Polu et al., 2023; Lample et al., 2022); using compiler-rejected tactics as negatives appears in BFS-Prover’s state-tactic DPO (Xin et al., 2025), trial-and-error fine-tuning (An et al., 2024), tactic-level verified rewards (Kim & Yun, 2026), and ImProver 2’s winner/loser rewrites. Our contribution on this axis is an empirical diagnosis: we show that negatives recorded in search order admit a trivial ranking shortcut (§4.2), and provide a complete-menu mode that removes it. The closest analogy outside theorem proving is learned compiler optimization, where models are trained on the output of symbolic/autotuning search (Cummins et al., 2023; 2025) or learn proposals for a verifier-gated rewrite search (Schkufza et al., 2013; Bunel et al., 2017; Shypula et al., 2024); LEANPOLISH instantiates this teacher-then-verified-student pattern for Lean proofs.

Lean datasets. LeanDojo (Yang et al., 2023), Lean Workbook (Ying et al., 2024), Goedel-Pset (Lin et al., 2025a), Herald (Gao et al., 2025) and FormalMATH (Yu et al., 2025) provide statements, proofs, or traces. LEANPOLISH instead aligns local spans and their verified replacements with available goal states and same-state attempted alternatives. This record structure supports both edit generation and controlled candidate ranking.

3 GENERATING VERIFIED EDIT SUPERVISION

LEANPOLISH turns a compiling proof into verified local improvement examples. It uses Lean’s InfoTree, which records the proof states produced while elaborating the original file. Candidate tactics can therefore be tested at the original state before re-checking the edited file. Four phases run in order: tactic replacement, local fact generalization, unused-fact removal, and unreachable-code cleanup. The key separation is between *proposal* (what to try), *policy* (which edits to prefer), and *verification* (whether the resulting proof still checks).

Tactic replacement. At every leaf tactic that closes a goal, LEANPOLISH tries a fixed menu in order (rfl, ring, abel, norm_num, norm_cast, positivity, decide, linarith, omega, field_simp, contradiction, ext, gcogr, tauto, simp? squeezed to simp only [...]), skipping any candidate that is not shorter than the original, with 5 s timeouts on the expensive tactics. The *first* candidate that closes the goal is proposed; if none does, a bounded exact? call is tried. A proposal is kept only if it passes a *specificity filter*: tactics that carry explicit terms, lemma names, or case structure supplied by the proof author (exact, rw, apply, refine,

use, cases, induction, calc, conv, ...) are never replaced unless the original is a multi-step block, and a decision procedure is never replaced by a more general one (e.g. `ring` only by `rfl`, `norm_num` never by `simp`). The filter encodes a maintainability preference; it is not a correctness mechanism (App. B).

Local fact generalization. Within one proof, near-duplicate `have` blocks are grouped and anti-unified; the differing subterms become parameters of one shared local fact, which is kernel-checked and substituted at each use. A merge is kept only if it saves bytes net of its own declaration and passes a *dependency filter*: the abstracted proof must depend on strictly fewer local free variables than the union of the original proofs, so that the parameters actually absorb local dependencies. This is a heuristic against merges that merely re-package the originals; it is not a guarantee of useful generalization, and its ablation leaves aggregate savings unchanged on the tested slices (§3.1). The filter applies to kernel-level extraction; exact duplicates and same-statement `have` blocks are instead replaced by a reference to the earlier hypothesis.

Unused facts and unreachable code. A usage graph identifies `have` blocks whose names are never referenced later; they are removed together, falling back to one at a time, and every removal is checked. When a file contains `<;>` chains, it is re-elaborated with Lean’s default linters (up to three rounds) and tactics reported as never executed or as doing nothing are removed from those chains.

Verification. Tactic candidates are tested against the saved proof state; the fully edited text is re-elaborated and kernel-checked in-process before it is saved. For the benchmark and competition sources (miniF2F, PutnamBench, Putnam 2025, and the frontier releases of §3.1), the saved file is additionally compiled by an independent `lake env lean` process from a fresh Mathlib import; if joint verification fails, a compiling subset of the edits is kept. Independently of the pipeline, the released verifier splices a single edit into its source file and compiles it in a fresh process; every model output counted as a success in §4 was checked this way. Edits touch only proof bodies, so theorem statements are unchanged. We do not claim that outputs are length-minimal or more readable to humans. The default toolchain is Lean 4.21.0 / Mathlib v4.21.0; frontier-source exceptions are specified in App. D.

Complete-menu search. Stopping at the first success leaves later alternatives unobserved. In *complete-menu* mode, LEANPOLISH continues through all 15 menu entries and selects the shortest successful candidate allowed by the specificity filter, breaking ties by menu order. It records the selected candidate, other valid candidates, Lean failures, timeouts, policy rejections, and candidates skipped for length separately. These are complete *search outcomes*; a timeout is not a proof of invalidity, and a policy rejection is not a correctness failure. The observed pools have 2.2–3.4 valid candidates per site, with at least two at 55–74% of sites, at 1.1–1.3× the menu-search time. Choosing a different one-token tactic can save characters without changing token counts; we observe no additional token compression from this mode. Its purpose is to remove first-success censoring, while retaining the declared menu and edit policy as part of the task (§4.2).

Fixed-point iteration. A single pass is not a fixed point: one edit can enable another (a tactic replacement can leave an earlier fact unused), and per-file time budgets cut some searches short. We therefore also run LEANPOLISH on its own output until no file changes (4–8 rounds, every round re-verified in a fresh process). This exposes compression that a one-pass baseline leaves available (§5).

What is recorded, and what the failures mean. Each accepted edit becomes a row with the available goal state (three renderings for tactic edits), the exact original and replacement spans with byte offsets, the edit kind, token/byte/line counts, and provenance (source file, toolchain and optimizer revisions). Each candidate that failed on the same goal in the same attempt becomes a linked *rejected sibling* with its error message. Because the menu is searched in a fixed order and stops at the first success, a sibling means “tried before the winner and failed” (an error or a timeout); candidates after the winner were never tried and are *unknown*, not negative. Siblings are recorded only for accepted edits; proposals rejected by the specificity filter and menu candidates skipped for not being shorter are not recorded. This matters for how the negatives can be used (§4.2).

Table 1: Released accepted edits and same-attempt failures per source. Token reduction is whole-corpus under the Lean-aware tokenizer (App. A), with files that are not shortened counted as 0%. Files = files with at least one accepted edit. Edit density and mean savings per edit are given in App. A.

Source (generator)	Inputs	Files	Accepted	Failed siblings	Tok. red. (%)
Mathlib v4.21.0 subset (human)	5,789	2,233	6,695	26,912	0.27
Goedel-Workbook (Goedel-Prover-V2)	29,750	10,052	20,822	28,525	5.48
PutnamBench sample (Goedel-Prover-V2)	437	352	4,354	5,930	8.14
miniF2F verified (Goedel-Prover-V2)	351	308	1,184	3,753	19.72
PutnamBench verified (Goedel-Prover-V2)	19	16	80	254	6.14
Putnam 2025 (AxiomProver)	12	10	142 / 125	147 / 75	1.31
Total		12,972 [‡]	33,402	65,596	

Putnam 2025 is processed under two scheduler configurations (sequential / pooled), shipped as separate shards and union-counted once in the file total; the displayed reduction uses the sequential configuration. [‡]Distinct files with an accepted edit, including 92 linter-baseline files (`*_linter.lean`) listed in the release; 12,880 are genuine inputs (App. H).

3.1 RELEASED RECORDS

Table 1 summarizes the release (public dataset; link in the Code and Data paragraph). About 36k files were processed; 12,972 received at least one accepted edit. The five sources span human-curated Mathlib, a large LLM-generated workbook (Goedel-Workbook proofs sampled from Goedel-Prover-V2-32B (Lin et al., 2025a;b)), two Goedel-Prover-V2 proof pools on benchmark statements (Zheng et al., 2022; Tsoukalas et al., 2024), and AxiomProver’s Putnam 2025 solutions (Axiom Math, 2025). By edit family, the 33,286 distinct accepted edits are 51.6% deletions of unused, no-op, or unreachable material, 47.6% tactic replacements, and 0.8% local fact generalizations.

Splits and leakage. Goedel-Workbook and Mathlib are training sources; miniF2F, PutnamBench-verified, and Putnam 2025 are held out. Goal-hash overlap between training and held-out sources is at most 0.15% Jaccard, and overlapping training rows are removed (App. F); deletion rows carry no goal and are separated by source only.

Release format. Accepted edits and failed siblings are linked JSONL streams with a Croissant manifest and datasheet; the repository adds a replay verifier and all scripts behind the tables (schema in App. G; errata in App. H).

Symbolic headroom across proof sources. In its release run, the symbolic pass removes 19.7% / 6.1% of tokens from Goedel-Prover-V2 proofs of miniF2F / PutnamBench-verified (27.5% / 16.1% when iterated to a fixed point), 5.5% on Goedel-Workbook, 1.3% on AxiomProver, 0.27% on Mathlib, and 2.6% on Seed-Prover 1.5’s Putnam 2025 release and 0.3–0.5% on Aristotle’s IMO 2025 files, versus $\leq 0.01\%$ for Mathlib’s `linter.unusedTactic` on identical inputs (App. A). Sources differ mainly in how often they contain unused facts and speculative tactic cascades (accepted edits per 1,000 tokens range 20 $\times$, savings per edit 4 $\times$); tactic replacement and unused-fact removal account for most savings, while local generalization is rare (0.8% of edits) and has no demonstrated aggregate compression benefit in these ablations.

4 SELECTION EFFECTS IN SEARCH-GENERATED SUPERVISION

A search that stops at its first success, and a benchmark built from the sites where it succeeded, both encode the search’s choices. We ask what a model trained on these records learns, and which evaluations can tell.

Task. Given the goal state at a site, a window of surrounding proof text, and the original span, a model must output a replacement span, or `<DELETE>` (prompt in App. E). Sites are those where LEANPOLISH (the *teacher*) found a verified edit on the held-out sources; this is *conditional local editing* at teacher-identified sites, not discovery of where to edit.

Table 2: Greedy editing at teacher-selected sites. “All” and “Tac.” are valid-and-shorter rates (%): 1,184 / 80 / 142 total sites and 747 / 43 / 57 tactic sites. “Tok.” sums individually verified savings; failures save zero. The verified *reference* can be byte-shorter without being token-shorter. The *delete rule* acts on the no-goal marker. *4-shot* uses the model’s chat template and four fixed examples (two replacements, two deletions). DPO starts from DeepSeek SFT.

Method	miniF2F			PutnamBench-verified			Putnam 2025 (AxiomProver)		
	All	Tac.	Tok.	All	Tac.	Tok.	All	Tac.	Tok.
Reference (pipeline)	94.6	91.6	27,622	88.8	79.1	1,359	84.5	61.4	1,746
Delete rule (no model)	36.8	0.0	8,122	46.3	0.0	975	59.9	0.0	1,500
DeepSeek, frozen	0.1	0.1	28	0.0	0.0	0	0.0	0.0	0
Qwen, frozen	0.5	0.8	189	0.0	0.0	0	1.4	0.0	32
DeepSeek, 4-shot	36.9	4.7	7,539	48.8	4.7	979	53.5	0.0	1,350
Qwen, 4-shot	46.9	17.9	9,046	48.8	7.0	964	59.2	12.3	1,401
DeepSeek, SFT	86.7	79.0	26,163	86.3	74.4	1,337	79.6	49.1	1,666
Qwen, SFT	84.5	75.5	25,317	82.5	67.4	1,316	76.8	42.1	1,648
DeepSeek, DPO	84.5	75.6	25,614	85.0	72.1	1,333	73.2	33.3	1,618

Data and models. From the 27,517 accepted edits of Goedel-Workbook and Mathlib we remove 9,842 exact duplicates (same goal, original, and replacement), 27 edits that are not shorter in bytes, and 112 rows whose goal hash occurs in a held-out source, and split the rest by source file into 17,099 training and 437 validation edits. We evaluate on all 1,406 held-out sites: 1,184 miniF2F, 80 PutnamBench-verified, and 142 Putnam 2025 (AxiomProver, a generator absent from training). We fine-tune DeepSeek-Prover-V2-7B (Ren et al., 2025) and Qwen2.5-Coder-7B-Instruct (Hui et al., 2024), a model not specialized for theorem proving, with identical BF16 LoRA (Hu et al., 2022) settings (rank 32, 2 epochs, under one H100-hour). Three training seeds per model differ by at most 1.4 points (standard deviation) on any held-out source.

Verification and metric. Each output is spliced into the original file at the site. A site counts as a success (*valid-and-shorter*) only if the spliced file received a pass verdict from a fresh, byte-exact compilation under Lean 4.21.0 / Mathlib v4.21.0 and the replacement is strictly shorter in tokens; reference-identical outputs are not assumed valid but are re-verified. Failures save zero tokens. We report greedy decoding; confidence intervals resample source files (10,000 replicates). Token totals here sum individually verified local edits; whole-file results in §5 instead verify and count the joint output. Zero-shot frozen models use the same raw prompt and decoding; four-shot controls additionally use each model’s chat template.

4.1 WHAT FINE-TUNING LEARNS AT TEACHER-SELECTED SITES

Fine-tuning improves conditional editing. Both frozen models almost never produce a valid shorter edit under greedy decoding, while both fine-tuned models reach 77–87% of all sites and recover 92–98% of the pipeline’s savings on the same sites (Table 2; DeepSeek SFT file-bootstrap 95% CIs [83.4, 89.8] / [75.8, 96.6] / [58.2, 95.2]). The benefit is also present without prover-specific pretraining: Qwen2.5-Coder, a general code model, is only 2.2 points lower on miniF2F (paired file bootstrap [0.8, 3.7]), and differences on the two smaller sources are not resolved. The near-zero zero-shot rates largely reflect the output interface. With each model’s chat template and four demonstrations, frozen models reach 37–59% of all sites, but almost entirely through deletions: they output <DELETE> at 72–78% of sites and succeed at only 0–18% of tactic-replacement sites, versus 42–79% after fine-tuning. These controls show that output formatting explains part of the zero-shot gap, while fine-tuning substantially improves tactic replacement over the tested prompts.

Separate deletions from tactic replacement. 558 of the 1,406 sites are deletions of unused or unreachable material, and at every one of them the prompt shows no local goal. A rule that deletes exactly at those sites, with no model, matches all 558 references (each with a pass verdict) and alone yields 8,122 / 975 / 1,500 tokens: on Putnam 2025, 90% of the fine-tuned model’s savings. Aggregate rates therefore mix a trivial decision with a harder one. Tactic replacement measures performance

beyond the deletion rule: the fine-tuned DeepSeek model succeeds at 79.0% / 74.4% / 49.1% of sites, against a reference that is itself token-shorter at 91.6% / 79.1% / 61.4%. Success is lower on AxiomProver proofs, whose generator and problem set both differ from the training sources; this comparison alone does not isolate the cause of the drop. The single held-out generalization site is never solved and is too small to evaluate learned abstraction.

Imitation, not improvement. Greedy exact match with the reference is 82.7% / 82.5% / 91.5% for DeepSeek SFT, and no fine-tuned output saves more tokens than its reference. Three training seeds per model reproduce these rates (standard deviation ≤ 1.4 points). DPO on accepted/failed pairs (Rafailov et al., 2023) trains stably but is 2.1 points below SFT on miniF2F (paired [1.0, 3.3]). At teacher-selected sites, the supervision teaches imitation of the teacher.

4.2 ORDERING LEAKAGE, AND COMPLETE POOLS THAT REMOVE IT

The failed siblings suggest a ranking task: given a goal and several candidates, pick the one that works. A ranker trained on 54,375 goal-disjoint pairs reaches 100% top-1 on all 718 held-out candidate groups, far above random (21.8%) and frozen log-probability (34.6%). This number is *not* evidence of mathematical discrimination. By construction (§3) each group contains the winner and the candidates tried *before* it, so choosing the candidate latest in the fixed menu order, using no goal, context, or model, also scores 100% on all 718 groups, because the winner is always the latest-tried candidate. The negatives thus encode the search policy, as in any first-success search that logs its failures.

Preference pairs inherit the same shortcut. This limits what their labels establish; our DPO comparison does not isolate it as the cause of DPO’s lower performance.

Complete pools. Complete-menu search (§3) removes this ordering shortcut. The target is the shortest valid, filter-passing candidate under the pool’s string-length criterion. We built pools for 6,641 held-out states (miniF2F, PutnamBench-verified, AxiomProver) and 53,272 states from Goedel-Workbook and Mathlib, and trained a pointwise ranker on 13,290 Goedel states, with no goal hash shared with the held-out pools. Table 3 evaluates the 2,388 held-out states whose pool contains at least one acceptable candidate (valid, strictly shorter, filter-passing); the other 4,253 states have no target and are excluded. A choice counts as best if it ties the shortest acceptable candidate; ties in a method’s scores are broken uniformly at random (in expectation). Re-running a subset at lower parallelism reproduced all 9,920 checked validity labels, providing evidence of label stability for that subset. On the evaluated pools (Table 3) the ordering rule that was perfect on the biased groups selects the best candidate 0% of the time, and model-free rules stay below 23%. Frozen models reach at most 36.9%, while the trained ranker reaches 70.1% (file-clustered 95% CI 66.3–73.9; validity AUROC 0.968) and recovers 79% of the oracle’s character savings. Removing the goal from its input costs 19.1 points (CI 15.2–22.7), significantly on miniF2F and PutnamBench but not on AxiomProver; the goal therefore supplies predictive information beyond the remaining input. First-success verified search selects the pool’s best candidate more often (76.9%). Exhaustive verification defines the pool oracle; the learned ranker’s accuracy does not establish a speed advantage or remove the need to verify its choice.

Table 3: Top-1 accuracy (%) for the shortest valid, filter-passing candidate on the evaluated complete pools (2,388 held-out states). *100% on the biased first-success groups.

Method	Top-1
Last in menu order*	0.0
First in menu order	11.6
Random	8.2
Shortest string	5.8
Per-tactic prior	22.6
Frozen Qwen-32B log-prob	28.3
Frozen DeepSeek-7B log-prob	36.9
Trained ranker, no goal	51.0
Trained ranker	70.1
Ref.: verifier, first success	76.9

5 FROM LOCAL SUPERVISION TO WHOLE-PROOF IMPROVEMENT

A local editor evaluated at teacher-selected sites inherits the teacher’s choice of where to act. We now remove that assistance: candidate sites are extracted from complete proofs, and savings are measured

Table 4: Whole-file token reduction (%) of Lean-verified final files, on specified input sets (Lean-aware counter; unshortened files count 0). miniF2F-99 is a fixed random 100-file subset of the 351 files, minus one linter-baseline artifact (App. H). *Release run*: the LEANPOLISH run that produced the dataset; *clean rerun*: the same binary re-run without resource contention. Hybrid rows: best of greedy and four samples; frozen models are DeepSeek-Prover-V2-7B unless stated. *filter*: only edits that pass LEANPOLISH’s specificity filter. LLM rewrites use $k=16$ samples.

Method	AxiomProver (12)	PutnamBench (19)	miniF2F-99
<i>Symbolic only</i>			
LEANPOLISH, release run	1.31	6.14	20.02
LEANPOLISH, clean rerun	1.41	11.91	20.52
LEANPOLISH (clean) iterated to a fixed point	1.76	16.07	29.76
<i>Neural only</i>			
LLM whole-proof rewrite, frozen	1.04 ^a	2.76	12.52
LLM rewrite, trained on LEANPOLISH pairs	— ^d	5.53	16.36
Local editor alone (trained)	2.51	3.10	—
<i>Hybrid (Alg. 2): release run, then verified local editor</i>			
frozen, few-shot	3.66	9.31	23.14
frozen, few-shot, filter	3.66	9.30	23.13
trained DeepSeek	3.65	9.02	22.41
trained DeepSeek, filter	2.38	8.04	21.37
trained Qwen	4.00	11.38	— ^c
<i>Composed pipelines</i>			
LEANPOLISH (release) → frozen LLM rewrite	2.34 ^a	7.88	27.58
LEANPOLISH (release) → trained LLM rewrite	— ^d	8.36	27.13
LEANPOLISH (clean) → frozen LLM rewrite	—	12.85	27.11
frozen LLM rewrite → LEANPOLISH	2.53 ^a	10.36	27.99
LEANPOLISH ↔ trained editor, alternated	5.05^b	20.25	—

^a $k=4$. ^bTwo rounds (PutnamBench: three). ^c23.76 on all 351 miniF2F files (DeepSeek: 21.74). ^dOnly 3 of 12 AxiomProver files fit the rewriter’s context; no gain on them.

on jointly verified output files. We ask whether neural proposals add beyond the symbolic search, and whether training is responsible for that gain.

Verified hybrid compression. Algorithm 2 runs LEANPOLISH, then asks a local editor (a trained or frozen 7B model, prompted without an explicit symbolic menu) for proposals at every remaining tactic site, verifies each in a fresh Lean process, combines compatible edits, and checks the resulting file again. This final check matters: edits that work separately may interfere when applied together.

5.1 SYMBOLIC FIXED POINT VERSUS NEURAL EDITS

Iterate the symbolic baseline before attributing neural gains. Table 4 compares complete output files rather than sums of local edits. On miniF2F-99, a single symbolic pass removes 20.0–20.5% of tokens, versus 12.5% for frozen whole-proof rewriting. Adding a local editor or whole-proof rewriter improves on that single pass. However, symbolic search also improves when given another opportunity: a clean rerun raises PutnamBench reduction from 6.1% to 11.9%, and iteration to a fixed point reaches 16.1% on PutnamBench and 29.8% on miniF2F-99 (27.5% on all 351 miniF2F files). The miniF2F fixed point exceeds every tested neural hybrid on the corresponding input set. A one-pass teacher therefore understates the available symbolic baseline.

Neural editing still adds on other sources. Alternating LEANPOLISH and the trained editor reaches 20.3% on PutnamBench, 4.2 percentage points above the symbolic fixed point, and 5.05% on AxiomProver, versus 1.76% symbolically. A single hybrid pass also improves six Seed-Prover 1.5 files from 4.37% to 6.36%. These establish additional compression in the tested configurations, not a compute-normalized advantage: the pipelines differ in model calls, verification work, and iteration count.

5.2 WHAT TRAINING CONTRIBUTES

Local editing: a strong frozen control. With the same sampling and verification loop, a frozen DeepSeek 7B editor with four demonstrations reaches 9.31% on PutnamBench, 23.14% on miniF2F-99, and 3.66% on AxiomProver, compared with 9.02%, 22.41%, and 3.65% for the trained editor. Thus these whole-file comparisons do not establish an advantage from fine-tuning, despite its large benefit at teacher-selected sites. The editors also act differently: 397 of the frozen editor’s 414 applied AxiomProver edits delete redundant tactic lines, whereas the trained editor makes substitutions and no deletions.

Compression also depends on the edit policy. About 97% of the trained editor’s accepted edits use tactics already in the symbolic menu; on AxiomProver, 526 of its 753 in-menu edits violate LEANPOLISH’s specificity filter, commonly replacing an explicit term by automation. Restricting the DeepSeek hybrid to filter-compliant edits reduces its AxiomProver savings from 3.65% to 2.38%, still above the 1.31% release baseline. The frozen editor’s deletions pass the same filter. These controls separate gains from broader proposals from gains obtained by relaxing the teacher’s policy. Qwen2.5-Coder trained on the same records yields higher hybrid reductions than DeepSeek on all three evaluated sources, so prover-specific pretraining is not necessary for this use.

Whole-proof rewriting: a benefit from the supervision. Fine-tuning DeepSeek-Prover-V2-7B for 67 GPU-minutes on 7,708 Goedel-Workbook proof pairs (original $\rightarrow$ LEANPOLISH output, 11% unchanged) raises verified token reduction at $k=16$ from 2.8% to 5.5% on the same 19 PutnamBench files. On miniF2F, the reported reductions on the same 99 files are 12.5% for the frozen rewriter and 16.4% after training, and 62% of the trained model’s samples yield a verified shorter file, against 16% for the frozen model (Table 4); theorem statements are unchanged. The trained rewriter remains below the symbolic fixed point, and training adds no consistent gain when rewriting follows LEANPOLISH (8.36% vs. 7.88% on PutnamBench; 27.13% vs. 27.58% on miniF2F-99). This supports transfer of the teacher’s edits to a whole-proof interface, without establishing improvement beyond the teacher’s remaining search space.

One round of self-training. We also test whether the editor’s own verified outputs improve it further (Polu et al., 2023; Ahuja et al., 2026). From 5,472 unchanged tactic sites in 1,000 training proofs, we obtain 372 distinct new edits; 147 violate the specificity filter. Continued training on either all new edits or filter-compliant non-deletions, mixed with replay, reduces success on the 847 held-out tactic sites from 74.9% to 72.9% or 70.7%, respectively (paired 95% CIs for the changes: $[-3.6, -0.5]$ and $[-6.1, -2.2]$ points). Whole-file savings beyond the teacher do not change significantly on PutnamBench or AxiomProver. This small, one-round experiment finds no benefit from further self-training; it does not establish a general limit on expert iteration (details in App. E).

6 DISCUSSION

LEANPOLISH makes proof-improvement supervision inspectable at the level where an edit is proposed and verified. The resulting resource exposes a distinction that aggregate accuracy obscures: a model can reproduce a search procedure’s choices without learning to improve on that search. Complete-menu outcomes remove one concrete shortcut, and the goal ablation shows useful state-dependent information in the remaining task. Whole-proof fine-tuning on PutnamBench provides a complementary positive result: symbolic edits can train a model to produce more effective verified rewrites.

The broader lesson is to evaluate the source of an improvement. A symbolic fixed point controls for edits missed by an early stop; a frozen editor controls for generate-and-verify alone; and a shared specificity policy distinguishes broader search from a different definition of an acceptable edit. These controls materially change our conclusions. They also give future work a concrete target: improve complete proofs beyond strong symbolic and prompted baselines at a declared computational budget.

The evidence has limits. Whole-file comparisons use 12–351 files per source and one trained checkpoint per configuration; three-seed checks cover conditional local editing only. Search remains bounded by its menu, imports, and time budgets, and complete pools do not remove all dataset selection effects. We measure length and compliance with a specificity policy, not human readability,

maintenance cost, or improved theorem-proving ability. Within this scope, LEANPOLISH contributes a verified neurosymbolic compression method, its supervision, and a protocol for testing what that supervision teaches. Verification certifies correctness; controlled comparisons show when learning helps.

CODE AND DATA

The code repository (<https://github.com/paulinebourigault/leanpolish>) contains the pipeline (including the complete-menu patch), the verifier, both token counters, and every script behind the tables, with a README mapping each table to its command and result files; all reported numbers are recomputed by these scripts from saved outputs over exact input-file lists. The dataset, the complete candidate pools, all model generations with their Lean verdicts, the verified output files, and the LoRA adapters of the editors are at <https://huggingface.co/datasets/leanpolish-anon/lean-proof-compression> (folders `experiments/` and `adapters/`). Toolchains are pinned (Lean 4.21.0, Mathlib v4.21.0); training hyperparameters and prompts are in App. E. Symbolic results can vary with machine speed through per-candidate time budgets (App. H); we report both the release run and a clean rerun.

REFERENCES

- Tudor Achim, Alex Best, Kevin Der, Mathis Fédérico, Sergei Gukov, Daniel Halpern-Leister, Kirsten Henningsgard, Yuri Kudryashov, Alexander Meiburg, Martin Michelsen, Riley Patterson, Eric Rodriguez, Laura Scharff, Vikram Shanker, Vladimir Sicca, Hari Sowrirajan, Aidan Swope, Matyas Tamas, Vlad Tenev, Jonathan Thomm, Harold Williams, and Lawrence Wu. Aristotle: IMO-level automated theorem proving, 2025. The Harmonic Team.
- Riyaz Ahuja, Jeremy Avigad, Prasad Tetali, and Sean Welleck. ImProver: Agent-based automated proof optimization. In *International Conference on Learning Representations*, 2025.
- Riyaz Ahuja, Tate Rowney, Jeremy Avigad, and Sean Welleck. ImProver 2: Iteratively self-improving LMs for neurosymbolic proof optimization, 2026.
- Chenyang An, Zhibo Chen, Qihao Ye, Emily First, Letian Peng, Jiayun Zhang, Zihan Wang, Sorin Lerner, and Jingbo Shang. Learn from failure: Fine-tuning LLMs with trial-and-error data for intuitionistic propositional logic proving. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. arXiv:2404.07382.
- Axiom Math. AxiomProver: An autonomous multi-agent ensemble theorem prover for Lean 4: Putnam 2025 solutions. GitHub repository, 2025. 12/12 problems on the Putnam 2025 competition. <https://github.com/AxiomMath/Putnam2025>.
- Rudy Bunel, Alban Desmaison, M. Pawan Kumar, Philip H. S. Torr, and Pushmeet Kohli. Learning to superoptimize programs. In *International Conference on Learning Representations (ICLR)*, 2017. arXiv:1611.01787.
- David M. Cerna and Temur Kutsia. Anti-unification and generalization: A survey. *Journal of Artificial Intelligence Research*, 78:293–361, 2023.
- Jiangjie Chen, Wenxiang Chen, Jiacheng Du, Jinyi Hu, Zhicheng Jiang, et al. Seed-Prover 1.5: Mastering undergraduate-level theorem proving via learning from experience, 2025.
- Chris Cummins, Volker Seeker, Dejan Grubisic, Mostafa Elhoushi, Youwei Liang, Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Kim Hazelwood, Gabriel Synnaeve, and Hugh Leather. Large language models for compiler optimization, 2023.
- Chris Cummins, Volker Seeker, Dejan Grubisic, Baptiste Roziere, Jonas Gehring, Gabriel Synnaeve, and Hugh Leather. LLM compiler: Foundation language models for compiler optimization. In *Proceedings of the 34th ACM SIGPLAN International Conference on Compiler Construction (CC)*, 2025. doi: 10.1145/3708493.3712691. arXiv:2407.02524.
- Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, pp. 625–635, 2021.

- Emily First, Markus N. Rabe, Talia Ringer, and Yuriy Brun. Baldur: Whole-proof generation and repair with large language models. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2023. doi: 10.1145/3611643.3616243.
- Yiming Fu, Peixuan Liu, Zichen Wang, and Kun Yuan. Proof-refactor: Refactoring generated formal proofs into modular artifacts, 2026.
- Guoxiong Gao, Yutong Wang, Jiedong Jiang, Qi Gao, Zihan Qin, Tianyi Xu, and Bin Dong. Herald: A natural language annotated Lean 4 dataset. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.10878.
- Alex Gu, Bartosz Piotrowski, Fabian Gloeckle, Kaiyu Yang, and Aram H. Markosyan. Proofoptimizer: Training language models to simplify proofs without human demonstrations. In *International Conference on Learning Representations*, 2026.
- Harmonic. Aristotle: Official lean 4 solutions to IMO 2025, 2025. Official formal-solution release.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.
- Thomas Hubert, Rishi Mehta, Laurent Sartran, Miklós Z. Horváth, Goran Žužić, Eric Wieser, Aja Huang, Julian Schrittwieser, et al. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, 651:607–613, 2025. doi: 10.1038/s41586-025-09833-y.
- Binyuan Hui, Jian Yang, Zeyu Cui, Jiayi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. Qwen2.5-coder technical report, 2024.
- Ruslan Khrulev. BlueprintRepair: Typed local edits for failed Lean proof blueprints, 2026.
- Minsu Kim and Se-Young Yun. Process-verified reinforcement learning for theorem proving via Lean, 2026.
- Guillaume Lample, Timothée Lacroix, Marie-Anne Lachaux, Aurélien Rodríguez, Amaury Hayat, Thibaut Lavril, Gabriel Ebner, and Xavier Martinet. HyperTree proof search for neural theorem proving. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- Yong Lin, Shange Tang, Bohan Lyu, Jiayun Wu, Hongzhou Lin, Kaiyu Yang, Jia Li, Mengzhou Xia, Danqi Chen, Sanjeev Arora, and Chi Jin. Goedel-prover: A frontier model for open-source automated theorem proving, 2025a.
- Yong Lin, Shange Tang, Bohan Lyu, Ziran Yang, Jui-Hui Chung, Haoyu Zhao, Lai Jiang, Yihan Geng, Jiawei Ge, Jingruo Sun, Jiayun Wu, Jiri Gesi, Ximing Lu, David Acuna, Kaiyu Yang, Hongzhou Lin, Yejin Choi, Danqi Chen, Sanjeev Arora, and Chi Jin. Goedel-prover-v2: Scaling formal theorem proving with scaffolded data synthesis and self-correction, 2025b.
- Haoxiong Liu, Jiacheng Sun, Zhenguo Li, and Andrew C. Yao. ProofAug: Efficient neural theorem proving via fine-grained proof structure analysis, 2025.
- Jialin Lu, Soonho Kong, Rodrigo Stehling, Kaiyu Yang, Zhangyang Wang, Weiran Sun, and Wuyang Chen. Lean refactor: Multi-objective controllable proof optimization via agentic strategy search, 2026.
- Azim Ospanov, Farzan Farnia, and Roozbeh Yousefzadeh. APOLLO: Automated LLM and Lean collaboration for advanced formal reasoning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2505.05758.
- G. D. Plotkin. A note on inductive generalization. In B. Meltzer and D. Michie (eds.), *Machine Intelligence 5*, pp. 153–163. Edinburgh University Press, 1970.
- Stanislas Polu, Jesse Michael Han, Kunhao Zheng, Mantas Baksys, Igor Babuschkin, and Ilya Sutskever. Formal mathematics statement curriculum learning. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2202.01344.

- Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Z. Z. Ren, Zhihong Shao, Junxiao Song, Huajian Xin, Haocheng Wang, Wanjia Zhao, Liyue Zhang, Zhe Fu, Qihao Zhu, Dejian Yang, Z. F. Wu, Zhibin Gou, Shirong Ma, Hongxuan Tang, Yuxuan Liu, Wenjun Gao, Daya Guo, and Chong Ruan. DeepSeek-Prover-V2: Advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition, 2025.
- Eric Schkufza, Rahul Sharma, and Alex Aiken. Stochastic superoptimization. In *Proceedings of the 18th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pp. 305–316, 2013. doi: 10.1145/2451116.2451150.
- Alexander Shypula, Aman Madaan, Yimeng Zeng, Uri Alon, Jacob Gardner, Milad Hashemi, Graham Neubig, Parthasarathy Ranganathan, Osbert Bastani, and Amir Yazdanbakhsh. Learning performance-improving code edits. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2302.07867.
- George Tsoukalas, Jasper Lee, John Jennings, Jimmy Xin, Michelle Ding, Michael Jennings, Amityay Thakur, and Swarat Chaudhuri. PutnamBench: Evaluating neural theorem-provers on the Putnam mathematical competition. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2024.
- Evan Wang, Simon Chess, Daniel Lee, Siyuan Ge, Ajit Mallavarapu, Jarod Alper, and Vasily Ilin. Learning to repair Lean proofs from compiler feedback, 2026. ICLR 2026 VerifAI Workshop.
- Haiming Wang, Mert Unsal, Xiaohan Lin, Mantas Baksys, Junqi Liu, Marco Dos Santos, Flood Sung, Marina Vinyes, Zhenzhen Ying, Zekai Zhu, Jianqiao Lu, Hugues de Saxcé, Bolton Bailey, Chendong Song, Chenjun Xiao, Dehao Zhang, Ebony Zhang, Frederick Pu, Han Zhu, Jiawei Liu, Jonas Bayer, Julien Michaud, Kaiyu Hu, Kashun Pernigo, Kjartan Kaarstan Liang, Lingxiao Wang, Mingfei Wu, Tao Hu, Tianyu Yan, Yifei Tsai, Abulay Aliyev, Bohdan Kushnirskyi, Igor Bohush, Boyang Yan, Yifan Zhu, Yujian Yu, Junyu Pan, Yongqian Yang, Yuhua Zhang, Zhengying Liu, and Jia Li. Kimina-Prover preview: Towards large formal reasoning models with reinforcement learning, 2025.
- Huajian Xin, Z. Z. Ren, Junxiao Song, Zhihong Shao, Wanjia Zhao, Haocheng Wang, Bo Liu, Liyue Zhang, Xuan Lu, Qiushi Du, Wenjun Gao, Qihao Zhu, Dejian Yang, Zhibin Gou, Z. F. Wu, Fuli Luo, and Chong Ruan. DeepSeek-Prover-V1.5: Harnessing proof assistant feedback for reinforcement learning and Monte-Carlo tree search, 2024.
- Jimmy Xin, Alex Schneidman, Chris Cummins, Karun Ram, Srihari Ganesh, and Jannis Limperg. AXLE: A cloud infrastructure for Lean 4 theorem proving utilities, 2026.
- Ran Xin, Chenguang Xi, Jie Yang, Feng Chen, Hang Wu, Xia Xiao, Yifan Sun, Shen Zheng, and Kai Shen. BFS-Prover: Scalable best-first tree search for LLM-based automatic theorem proving. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025. arXiv:2502.03438.
- Kaiyu Yang, Aidan Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan J. Prenger, and Anima Anandkumar. Leandojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Huaiyuan Ying, Zijian Wu, Yihan Geng, Jiayu Wang, Dahua Lin, and Kai Chen. Lean workbook: A large-scale Lean problem set formalized from natural language math problems. In *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 105848–105863, 2024.
- Zhouliang Yu, Ruotian Peng, Keyi Ding, Yizhe Li, Zhongyuan Peng, Minghao Liu, Yifan Zhang, Zheng Yuan, Huajian Xin, Wenhao Huang, Yandong Wen, Ge Zhang, and Weiyang Liu. Formal-MATH: Benchmarking formal mathematical reasoning of large language models, 2025.
- Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. MiniF2F: A cross-system benchmark for formal olympiad-level mathematics. In *International Conference on Learning Representations (ICLR)*, 2022.

APPENDIX

A Symbolic Compression Across Proof Sources	13
B Method Details	15
C Symbolic Results: Full Tables	16
D Frontier-Prover Details	16
E Learning Experiment Details	17
F Leakage Audit	18
G Release Schema	18
H Data Notes	19
I Using LeanPolish and the Release	20
J Worked Examples	21

A SYMBOLIC COMPRESSION ACROSS PROOF SOURCES

This section supports the cross-source results in §3.1: it defines the compression metric, distinguishes matched baselines from published contextual comparisons, and interprets the phase ablations.

Metric. We count tokens with a Lean-aware counter following the rules of Appendix L of [Gu et al. \(2026\)](#): identifiers, literals, and multi-character operators count as one token, and whitespace and comments are skipped. This is the counter LEANPOLISH uses internally; our released Python port reproduces its per-file counts exactly. For an input set $\mathcal{F}$, whole-corpus token reduction is $100 \sum_{f \in \mathcal{F}} [L(f) - L(\hat{f})] / \sum_{f \in \mathcal{F}} L(f)$, where L is the token count and $\hat{f} = f$ when no shortened output is accepted. This weights files by original length; it is not the mean of per-proof percentage reductions. Exact input-file lists are released. A second released tokenizer that also counts comments gives similar numbers except on comment-heavy sources (Goedel-Workbook: 9.5% instead of 5.5%, because removed proof text often carries comments); both counters are released, and results using the second counter are identified explicitly. We also report bytes and non-blank lines (App. C); they agree on the extremes but rank Goedel-Workbook higher, because removed text there often carries comments.

Matched-input comparison with the linter. Mathlib’s `linter.unusedTactic` flags top-level tactics whose removal leaves the proof state unchanged; it is the symbolic first stage of ProofOptimizer, and the only published component we can run on identical inputs. We reproduce its configuration and apply both tools to the same files, toolchain, and tokenizer (Table 5). On Goedel-Prover-V2 miniF2F and PutnamBench proofs, LEANPOLISH removes 19.7% and 6.1% of tokens versus $\leq 0.002\%$ for the linter; on Mathlib, where the linter already runs in CI, both are near zero. ProofOptimizer reports 9.2% / 7.4% for the linter on 194 / 75 Goedel-Prover-V2 proofs under Mathlib 4.19; under our v4.21 configuration it rarely fires on our 351 / 19 proofs. We have not isolated whether the proof samples or the Mathlib version account for this difference.

Table 5: Whole-corpus token reduction (%) on identical inputs, toolchain, and tokenizer. ProofOptimizer’s figures are published per-proof average reductions on its own Goedel-Prover-V2 inputs for the same benchmarks, shown for reference only (not a head-to-head comparison).

Input proofs (files shortened / total)	LEANPOLISH	unusedTactic linter	ProofOptimizer (ref.)
miniF2F, Goedel-Prover-V2 (308/351)	19.7	0.002	87.9
PutnamBench, Goedel-Prover-V2 (16/19)	6.1	0.000	57.2
Goedel-Workbook	5.48	0.010	—
Mathlib v4.21.0 subset	0.27	0.000	—
Putnam 2025, AxiomProver	1.31	0.000	—

Table 6: Descriptive edit statistics for the release run in Table 1. Density is accepted edits per 1,000 original tokens; tok/edit is mean tokens saved per accepted edit. Putnam 2025 uses the sequential configuration. These summarize the edit mix, not a causal model of compression.

Source	Density	Tok/edit
Mathlib v4.21.0 subset	0.46	5.9
Goedel-Workbook	4.72	11.6
PutnamBench sample	9.10	8.9
miniF2F verified	8.45	23.3
PutnamBench verified	3.62	17.0
Putnam 2025 (AxiomProver)	1.06	12.3

Frontier-prover releases. To test whether symbolic headroom persists on proofs from newer agentic systems, we ran the unchanged pipeline on the official Seed-Prover 1.5 Putnam 2025 solutions (Chen et al., 2025) and Aristotle’s IMO 2025 solutions (Harmonic, 2025), without modifying any input. Seed-Prover pins Lean/Mathlib 4.22, so for a matched comparison we ran both Seed-Prover and AxiomProver under a common 4.22 toolchain on the eight problems where both releases compile; the port changes one heartbeat option in the optimizer and the per-file time budget, not the inputs. Seed-Prover proofs shrink by 2.7% (3,803 / 139,490 tokens) versus 1.2% (966 / 80,107) for AxiomProver, with per-problem reductions from 0% to 6.2% (App. D). Under each release’s native toolchain, all eleven Seed-Prover files give 2.55%, and the full AxiomProver release gives 1.31% (Table 1). Aristotle’s two processable files shrink by 0.3% and 0.5%; one file is toolchain-incompatible and one 103 KB file triggers a deterministic indexing failure in the optimizer, which we report as a robustness limitation. These are small samples and independent formalizations of the same problems, so we claim no trend; they show that headroom on frontier outputs is real but modest.

Why reductions vary. For non-overlapping edits whose savings sum to the file-level change, accepted-edit density (per 1,000 original tokens) times mean tokens saved per edit, divided by 1,000, gives the reduction as a fraction. These descriptors help interpret the file-level metric; overlapping local edits must not be summed as a whole-file result. Density spans $20\times$ across sources (0.46 to 9.10 edits per 1,000 tokens; Table 6) while mean savings per edit spans $4\times$ (5.9 to 23.3 tokens); both matter (PutnamBench-verified has lower density than Goedel-Workbook but larger edits, and a larger reduction). For example, Mathlib’s 6,695 edits touch 2,233 of 5,789 files at 5.9 tokens each, whereas miniF2F’s 1,184 edits touch 308 of 351 files at 23.3 tokens each. This is a descriptive decomposition, not a causal account, and is consistent with the observed edit mix: tactic replacement and removal of unused material account for most measured savings.

Which phases matter. On a random 500-file Goedel-Workbook slice, disabling tactic replacement lowers token savings from 6.58% to 3.86%; disabling unused-fact removal or cleanup costs about 1.2 points; disabling generalization or its dependency filter leaves token savings essentially unchanged (6.61%, 6.58%). On a slice selected for many `have` blocks, unused-fact removal dominates (9.08% $\rightarrow$ -0.23%), and disabling generalization slightly *increases* token savings (9.53%). Generalization is thus a rare edit family (0.8% of accepted edits) whose value we do not establish by compression; disabling the specificity filter shortens more files (180 vs. 167) without increasing token savings (full table in App. C).

Algorithm 1 One LEANPOLISH pass, with first-success or complete-menu search (§3)**Input:** compiling file F ; ordered tactic menu $\mathcal{M}$; specificity filter Q ; mode $\in \{\text{FIRST}, \text{COMPLETE}\}$.

1. Elaborate F once; collect goal-closing tactic sites s (non-structural nodes longer than 3 bytes) with their goals g_s .
2. For each site s : *first*: try $\mathcal{M}$ in order and stop at the first c that closes g_s and is shorter than s ; *complete*: try all of $\mathcal{M}$, log every outcome, and take the shortest valid, shorter c (ties: menu order). Keep c only if $Q(s, c)$ holds (else no edit); if $\mathcal{M}$ yields nothing, try a bounded `exact?`. Record (s, g_s, c) and the failed siblings.
3. Add local-fact generalization, unused-have and unreachable-code candidates.
4. Select non-overlapping candidates greedily by byte savings; re-elaborate the edited file, blame errors on candidates, and re-select (up to 4 rounds).
5. Clean `<;>` chains with Lean’s linters (up to 3 rounds), then re-check the output in a fresh `lake env lean` process.
6. Return the verified file and its accepted and failed records. Iterating passes until no file changes gives the fixed point.

B METHOD DETAILS

This section specifies the policy and generalization checks used in §3. They govern which verified edits the symbolic teacher prefers; they are distinct from the final correctness check. Algorithm 1 summarizes one pass of LEANPOLISH.

Specificity filter. The filter groups tactics into tiers: `rfl`; `ring` and `abel` (replaceable only by `rfl`); `norm_num`, `positivity`, `norm_cast`, `decide` (interchangeable, never replaced by lower tiers); `linarith`, `nlinarith`, `omega` (peers); `field_simp`, `gcongr`, `tauto` (never replaced by `simp`). `trivial`, `assumption`, and `contradiction` are never replaced, and no structural tactic is replaced by a search-based one (`simp`, `tauto`, `aesop`). Witness tactics (`exact`, `rw`, `apply`, `refine`, `use`, `convert`, `cases`, `rcases`, `obtain`, `calc`, `induction`, `match`, `constructor`, `left`, `right`, `conv`, ...) are never replaced unless the original is a multi-step block (`;` or `<;>`). This tier structure is distinct from the *search* order of §3: the search proposes the first successful candidate, and the filter then accepts or rejects that single proposal. Additional guards reject rewrites of non-Prop goals, goals closed through `sorryAx`, and `exact?` suggestions whose head constant is not from the imported library (e.g. the theorem being proved).

Dependency filter for generalization. For a group of N near-duplicate `have` blocks with proof terms $p_1, \dots, p_N$, let U be the set of local free variables occurring in any p_i , and let q be the anti-unified proof after abstracting the differing subterms into parameters (via `mkForallFVars/mkLambdaFVars`, checked with `Meta.check` and `isDefEq`). The merge is rejected if $|\text{FV}(q)| \geq |U|$ (logged as `g3_no_generalization_ageb`). This filter is a heuristic; we make no information-theoretic claim for it, and a count of parameters or free variables alone does not determine whether a merge shortens the proof. Table 7 gives the rule’s behavior on a stratified sample.

Table 7: Local fact generalization on a stratified random sample (seed 42).

	Goedel-Workbook	Mathlib v4.21.0
Files sampled	1,500	250
Candidate groups discovered	86	49
Reached the dependency filter	16	1
Rejected by dependency filter	6	0
Applied	10	1

Complexity and runtime. Per file, tactic replacement makes $O(T)$ bounded Lean calls for T leaf tactics; unused-fact removal builds a usage graph linear in the number of local hypotheses and needs at most one check per block; generalization compares blocks only within signature buckets. Each Lean process loads Mathlib once and processes files sequentially; a Python orchestrator parallelizes

across processes. The full 29,750-file Goedel-Workbook run takes about one hour on a 96-core machine; 309 files (1.0%) fail (timeout or error) and are counted as unshortened.

C SYMBOLIC RESULTS: FULL TABLES

These tables give the alternative size metrics and phase ablations behind §3.1 and App. A. They separate the source of compression from the mere presence of a phase in the pipeline.

Table 8: Whole-corpus reduction (%) in bytes (B), tokens (T), and non-blank lines (L); files not shortened count as 0%.

Corpus	LEANPOLISH			linter.unusedTactic		
	B	T	L	B	T	L
miniF2F (Goedel-V2 verified)	19.83	19.72	21.59	—	0.002	—
PutnamBench (Goedel-V2 verified)	6.22	6.14	7.29	—	0.000	—
Mathlib v4.21.0	0.24	0.27	0.18	0.000	0.000	0.000
Goedel-Workbook	10.66	5.48	12.05	0.018	0.010	0.000
PutnamBench (Goedel sample)	8.82	8.14	11.39	0.013	0.007	0.000
Putnam 2025 / AxiomProver	1.21	1.31	1.20	0.000	0.000	0.000

Linter reproduction. We enable `linter.unusedTactic`, disable `linter.unreachableTactic` and `linter.unnecessarySeqFocus`, fix `maxHeartbeats` at 800,000, widen detected ranges to trailing separators, splice them out, and re-elaborate. ProofOptimizer reports its linter figures on 194 / 75 Goedel-Prover-V2 proofs under Mathlib 4.19; we apply the configuration to 351 / 19 Goedel-Prover-V2 proofs under v4.21; we have not isolated whether the proofs or the Mathlib version explain why the linter rarely fires here.

Table 9: Phase ablation on Goedel-Workbook. (a) Random 500-file slice; (b) 500 files selected for many `have` blocks. Reduction in %; Gen = applied generalizations; Filt = generalizations rejected by the dependency filter. Identical aggregate rows indicate no measured change; phase interactions can offset individual edits.

Configuration	Shorter	B	T	L	Gen	Filt
<i>(a) random slice</i>						
Full pipeline	167	11.75	6.58	9.90	2	3
– tactic replacement	108	1.94	3.86	6.31	2	3
– generalization	166	11.74	6.61	9.88	0	0
– unused-fact removal	151	11.31	5.40	8.66	2	3
– cleanup	131	11.06	5.34	8.09	2	3
– specificity filter	180	11.75	6.58	9.81	2	3
– dependency filter	167	11.75	6.58	9.90	2	0
<i>(b) have-rich slice</i>						
Full pipeline	245	5.92	9.08	10.64	57	15
– tactic replacement	240	5.75	9.07	10.64	60	14
– generalization	240	5.70	9.53	10.31	0	0
– unused-fact removal	44	0.51	−0.23	0.60	57	2
– cleanup	245	5.92	9.08	10.64	57	15
– specificity filter	254	5.44	8.67	9.83	56	16
– dependency filter	245	5.92	9.08	10.64	57	0

D FRONTIER-PROVER DETAILS

This section supports the scope of the frontier-source comparison in §3.1. It records the shared-toolchain subset and failures so that source differences are not mistaken for a controlled comparison of prover quality.

Table 10: Seed-Prover 1.5 vs. AxiomProver Putnam 2025 solutions under a shared Lean/Mathlib 4.22 toolchain, on the eight problems where both releases compile. Tokens removed / original tokens.

Problem	Seed-Prover 1.5	AxiomProver
A1	371 / 9,105 (4.1%)	118 / 7,067 (1.7%)
A2	0 / 6,115 (0.0%)	72 / 5,385 (1.3%)
B1	0 / 12,039 (0.0%)	115 / 15,590 (0.7%)
B2	1,702 / 27,478 (6.2%)	64 / 5,502 (1.2%)
B3	0 / 6,063 (0.0%)	76 / 3,552 (2.1%)
B4	230 / 8,503 (2.7%)	393 / 13,360 (2.9%)
B5	0 / 34,371 (0.0%)	128 / 16,575 (0.8%)
B6	1,500 / 35,816 (4.2%)	0 / 13,076 (0.0%)
Total	3,803 / 139,490 (2.7%)	966 / 80,107 (1.2%)

Algorithm 2 Verified hybrid compression (§5)

Input: compiling file F , editor M , number of proposals k .

1. Run the verified symbolic pass: $F_1 \leftarrow \text{LEANPOLISH}(F)$.
 2. Extract tactic sites S and their goal states from F_1 ; set $E = \emptyset$.
 3. For each $s \in S$, generate k proposals and retain only shorter spans, forming C_s .
 4. Verify each splice independently: $V_s = \{c \in C_s : \text{COMPILES}(F_1[s \mapsto c])\}$. If $V_s \neq \emptyset$, add a candidate with maximum savings and its site to E .
 5. Apply a compatible, non-overlapping subset of E and verify the whole file. If joint verification fails, reduce the subset; use F_1 if no subset succeeds.
 6. Return the jointly verified file and measure its savings relative to F .
-

Inputs are the unmodified official releases (file hashes recorded in the code repository). The 4.22 port adds a single `maxHeartbeats` option to the optimizer (not to the inputs) and raises the per-file budget to 7,200 s; the algorithm, phases, filters, and tokenizer are unchanged. Under 4.22, 11/11 Seed-Prover files and 8/12 AxiomProver files compile; where a problem was run under both toolchains, savings agree exactly on three of five problems and closely on the other two. Seed-Prover B2 completed only under 4.22 (it exceeded a 14,400 s budget under 4.21). Aristotle’s IMO 2025 release pins Lean 4.20-rc5; P3 and P5 are processed end to end (0.3%, 0.5%), one file does not compile under our toolchain, and P4 (103 KB) triggers a deterministic out-of-range indexing error in the optimizer.

E LEARNING EXPERIMENT DETAILS

This section separates the learning tasks in §4 from the whole-file tests in §5. Conditional editing predicts a span at a teacher-selected site; complete-pool ranking selects among logged alternatives; whole-proof rewriting generates an entire proof. Their success rates have different denominators.

Prompt. You are editing a Lean 4 proof. Return only a shorter replacement that preserves the goal. Return <DELETE> if the fragment should be removed. followed by [GOAL] (the goal state, or “(no local goal)”), [LOCAL CONTEXT] (surrounding source), [ORIGINAL FRAGMENT], and [REPLACEMENT].

Local-editor training and decoding. LoRA rank 32, $\alpha = 64$, dropout 0.05 on all linear layers; 2 epochs; learning rate 10^{-4} with cosine schedule and 3% warmup; weight decay 0.01; effective batch 64; seed 42; BF16 on one H100. Decoding: greedy, plus 4 samples at temperature 0.6, top- p 0.95 for best-of-4 analyses.

Verification. Candidates are first screened in a persistent Lean REPL; every candidate counted as a success then receives an individual verdict from a fresh `lake env lean` compilation of the spliced file, including reference-identical outputs. The original local-editor verification batch contains

2,086 distinct candidate splices with an exact verdict, with no conflicting verdicts and full agreement with the screen.

Selection among sampled outputs. Pooled verified savings over the three held-out sources when choosing among four SFT samples: greedy 29,166 tokens; random choice 28,832 (expectation); learned ranker 28,720; shortest string 28,354; verifier oracle 29,250. The ranker selects fewer savings than random choice.

Complete-pool ranker, further splits. On the in-distribution Goedel-Workbook validation pools the ranker reaches 80.7% top-1 best (no goal: 68.8%; validity AUROC 0.994); on a 3,000-state sample of human-written Mathlib proofs it reaches 53.8% (no goal: 41.3%). On the Mathlib sample, first-success verified search (menu order, stop at the first acceptable candidate) reaches 91.4%. Ranker: DeepSeek-Prover-V2-7B with LoRA ($r=16$), two pointwise heads (valid, best), trained for one epoch on 80,343 candidates, whole sites subsampled (seed 42) from 191,321 candidate rows of 13,290 Goedel training states.

DPO. One epoch on 16,839 same-attempt pairs (chosen = accepted edit, rejected = failed sibling), initialized from the DeepSeek SFT model (LoRA $r = 16$, $\alpha = 32$, learning rate 10^{-5} , $\beta = 0.1$), same verification; results are in Table 2. Because DPO pairs inherit the search-order structure of §4.2, we treat DPO as a check of the preference interface, not as an improvement method.

Whole-proof training and self-training. The whole-proof rewriter uses 7,708 original/output Goedel-Workbook pairs, including 11% unchanged pairs, and 67 GPU-minutes of training; evaluation uses $k = 16$ except where Table 4 states otherwise. These experiments use one checkpoint per configuration. The prompt is “Rewrite this Lean 4 proof to be shorter while remaining correct. Output the full file.” followed by the file. Pairs come only from files in the seed-42 file-level SFT training split (validation files and goal-overlap drops excluded), are rebuilt byte-exactly from the edit shards (360 failing reconstruction checks dropped), and fit an 8,192-token budget (max 3,868). Training uses LoRA on DeepSeek-Prover-V2-7B ($r=32$, $\alpha=64$, dropout 0.05, attention and MLP projections), learning rate 10^{-4} (cosine, 3% warm-up), two epochs, loss on the output only, 467 steps. Sampling uses $T=0.7$, top- p 0.95, seed 42; the sample rate is the fraction of all samples that yield a verified, strictly shorter file. For local-editor self-training, 410 shorter proposals from 5,472 sites compile, leaving 372 after deduplication. Only two are deletions; 147 (40%) violate the specificity filter. The new edits save 0.55% of input tokens under the comment-counting counter. We compare continued training on all new edits with training on filter-compliant non-deletions, each mixed with a fixed random replay sample (seed 42) of SFT training rows equal in size to the naive new-edit set (744 and 595 rows in total). Both continue from the round-1 adapter for one epoch with the round-1 recipe (learning rate 10^{-4} cosine, 3% warm-up, weight decay 0.01, effective batch 64, maximum length 2,048). The update comprises only 10–12 optimizer steps; a from-base retrain also finds no gain. Filter compliance is 96% at teacher-selected sites, 32% on the tested non-teacher sites, and 34% after naive self-training. These observations motivate the limited interpretation in §5.2.

F LEAKAGE AUDIT

This audit supports the split in §4. Hashes compare whitespace-normalized goal strings, not semantic equivalence; low exact overlap does not rule out related statements or overlap in model pretraining. The PutnamBench-sample rows below describe the release but are not a training source for the local editors. Non-goal deletion rows are separated by source rather than by placeholder hashes.

G RELEASE SCHEMA

The schema implements the local supervision record in §3. Consumers should distinguish original first-success siblings from the complete-menu outcomes illustrated in App. I.

Accepted rows (schema_version 2) expose the following fields where applicable: original, replacement, goal_state, goal_pretty, goal_type, type (edit family), kind (Lean syntax kind), start_byte, end_byte, line, byte/token/line counts of both spans,

Table 11: Goal-hash overlap between training and held-out sources (unique whitespace-normalized goal hashes; Jaccard is a fraction).

Train $\cap$ held-out	Train	Held-out	$\cap$	Jaccard
Goedel $\cap$ miniF2F	5,193	696	9	0.0015
Goedel $\cap$ PutnamBench-verified	5,193	38	1	0.0002
Goedel $\cap$ Putnam 2025	5,193	55	1	0.0002
Mathlib $\cap$ miniF2F	6,490	696	2	0.0003
Mathlib $\cap$ PutnamBench-verified	6,490	38	1	0.0002
Mathlib $\cap$ Putnam 2025	6,490	55	2	0.0003
PutnamBench sample $\cap$ miniF2F	561	696	4	0.0032
PutnamBench sample $\cap$ PutnamBench-verified	561	38	8	0.0135
PutnamBench sample $\cap$ Putnam 2025	561	55	1	0.0016

edit_width (UTF-8 byte difference), savings, term_size, context, file, corpus, attempt_id, rank_in_attempt, outcome, failed_tactics, failed_attempts (tactic, error, wall time), and provenance (optimizer revision as git_sha or commit_sha, mathlib_rev, content_sha256). Rejected rows share the same fields plus err_msg and wall_ms, with outcome rejected_attempt; finer failure reasons (kernel error, unsolved goals, timeout, parse error) are derived from err_msg and are not a separate label. For non-tactic rows goal_type holds a family placeholder and goal_state is empty; these fields must not be used for goal-hash deduplication of such rows. For strict compression training we recommend keeping rows with positive edit_width (33,360 of 33,402).

H DATA NOTES

These notes qualify the release counts in Table 1, byte-exact replay, and the comparison denominators in Table 4.

(i) *Hashes*. In the miniF2F and PutnamBench-verified shards, content_sha256 is the hash of the working file when the row was emitted, which differs from the input file once earlier edits in the same file have been applied; edit spans are byte-exact against the input files, and the replay verifier ignores this field for these shards. (ii) *License*. The release is distributed under Apache-2.0 (dataset card and Croissant manifest); upstream licenses are listed per source. (iii) *Units*. edit_width is in UTF-8 bytes, not characters. (iv) *Linter-baseline files*. The Goedel-Workbook and PutnamBench-sample shards contain 92 files named *_linter.lean (133 edits), which are outputs of the linter baseline rather than optimizer inputs; they are listed in the release. They are not used as optimizer inputs in any evaluation; the one such file in the random miniF2F-100 subset is removed, so every miniF2F row of Table 4 uses the same 99 files. (v) *Denominators*. All reductions are computed over exact input-file lists, released in the code repository. (vi) *Machine dependence*. LEANPOLISH outcomes depend on machine speed through per-candidate and per-file time budgets: a clean rerun of the release binary shortens PutnamBench-verified by 11.9% instead of 6.1% (the release run hit budgets on several files), and a single pass is not a fixed point. We report both runs. The local-editor hybrid block starts from the release run; composed-pipeline rows explicitly identify clean reruns.

I USING LEANPOLISH AND THE RELEASE

These recipes connect the resource in §3 to the local learning and whole-file experiments in §4–5. The pool example makes the first-success shortcut concrete.

Recipes. All commands are in the code repository (its README lists every script and the table it reproduces).

1. *Build* (Lean 4.21.0, Mathlib v4.21.0): `lake exe cache get && lake build LeanPolish`.
2. *Compress a folder of proofs*: `python3 leanpolish.py --batch DIR --workers 6 --threads 4 --timeout 1800 writes *_shortened.lean`, a per-file report, and the accepted and failed edits; use the independent verifier below for byte-exact replay.
3. *Fixed point*: re-run step 2 on the shortened outputs until no file changes (`complete_menu/fixpoint.py` automates it; 4–8 rounds in our corpora).
4. *Complete candidate pools*: add `--complete-menu`; each site then lists all 15 menu entries, including explicit length skips, (format on the dataset card; example in Table 12).
5. *Verify any edit independently*: `verify_pair.py ROWS.jsonl --project-root DIR` splices each edit into its source file and compiles it in a fresh process.
6. *Train and run a local editor*: `build_sft_data.py` $\rightarrow$ `train_sft_lora.py`; the verified hybrid of Algorithm 2 is `beyond_teacher/scripts/pipeline.sh`; both LoRA adapters are released.

What a complete pool records. Table 12 shows one held-out state. First-success search stops at `linarith`, so the first-success release would store the seven earlier failures as its only negatives and never test `omega`; the complete pool shows four valid candidates, a character-shorter best one, and a label for every entry. The valid one-word tactics all have the same token count: this example illustrates a ranking distinction, not an additional token saving.

Table 12: One complete pool (PutnamBench `putnam_1975_a1`). Goal $\uparrow n = (\uparrow a^2 + \uparrow a)/2 + (\uparrow b^2 + \uparrow b)/2$; original span `simpα [add_assoc] using h₁`. Entries in menu order; outcomes as released.

Candidate	Outcome	Note
<code>rfl, ring, abel, norm_num, norm_cast, positivity, decide</code>	kernel failure	tried and failed: the only negatives a first-success release stores
<code>linarith</code>	valid, not chosen	first-success winner (8 characters)
<code>omega</code>	chosen	shortest valid candidate (5 characters)
<code>field_simp, contradiction, ext, simp?</code>	kernel failure	never tried by first-success search
<code>gcgr, tauto</code>	valid, not chosen	never tried by first-success search

J WORKED EXAMPLES

The examples illustrate the proposal families of §3 and the policy distinction tested in §5.2. Ellipses mark abridged excerpts; the displays are not standalone Lean files, and numerical savings refer to the complete recorded edits.

Symbolic edits. The excerpts illustrate, in order: a goal closed by `simp` that is definitional, a speculative tactic cascade that `ring` alone closes, and combinator arms that run after the goal is already closed.

Tactic replacement (`simp` → `rfl`) lean_workbook_10208.lean

```
| zero =>
-   simp
+   rfl
```

Cascade collapsed to `ring` lean_workbook_1001.lean

```
- := by
-   simp only [sq, mul_add, mul_comm, mul_left_comm, mul_assoc,
-             add_assoc, add_left_comm]
-   ring
-   <|> simp [Complex.I_sq]
-   <|> ring
+ := by ring
```

Unreachable-code cleanup lean_workbook_plus_61403.lean

```
ring_nf
- <|> simp_all only [sub_eq_add_neg]
- <|> ring_nf
- <|> simp_all only [sub_eq_add_neg]
- ... (22 more unreachable arms)
```

Verified neural edits (hybrid, §5). The full versions of all three edits are accepted by Lean. The first two also pass LEANPOLISH’s specificity filter (pruning unneeded hints; dropping a type the elaborator infers, an edit outside the symbolic menu). The last does not: it replaces an explicit proof term by automation, illustrating the policy relaxation in the trained hybrid.

Hint pruning (compliant; −37 tokens) PutnamBench putnam_1993_a2

```
- nlinarith [sq_pos_of_ne_zero (xnonzero k), ... (4 hints)]
+ linarith
```

Redundant type dropped (compliant; −30 tokens) miniF2F

```
- have h4 : f (-2 + 3) = 3 * (-2 : ℝ) ^ 2 + 7 * (-2 : ℝ) + 4 := h0
  (-2)
+ have h4 := h0 (-2)
```

Term → `tauto` (filter-rejected; −27 tokens) AxiomProver

```
- exact ⟨fun ⟨h1, h2⟩ => ⟨h2, h1⟩, fun ⟨h1, h2⟩ => ⟨h2, h1⟩⟩
+ tauto
```